

PROGRAM:

```
#include <stdio.h>

#include <stdlib.h>

#define MAX_VERTICES 100

struct Node {

    int vertex;

    struct Node* next;

};

struct Graph {

    struct Node* adjLists[MAX_VERTICES];

    int visited[MAX_VERTICES];

};

// Create a node

struct Node* createNode(int v) {

    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->vertex = v;

    newNode->next = NULL;

    return newNode;

} // Create a graph

struct Graph* createGraph() {

    struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));

    int i;

    for (i = 0; i < MAX_VERTICES; i++) {

        graph->adjLists[i] = NULL;

        graph->visited[i] = 0;

    }

    return graph;

}
```

```

// Add edge

void addEdge(struct Graph* graph, int src, int dest) {

    // Add edge from src to dest

    struct Node* newNode = createNode(dest);

    newNode->next = graph->adjLists[src];

    graph->adjLists[src] = newNode;

    // Add edge from dest to src

    newNode = createNode(src);

    newNode->next = graph->adjLists[dest];

    graph->adjLists[dest] = newNode;

}

// Print the graph

void printGraph(struct Graph* graph) {

    int v;

    for (v = 0; v < MAX_VERTICES; v++) {

        struct Node* temp = graph->adjLists[v];

        printf("\n Adjacency list of vertex %d\n ", v);

        while (temp) {

            printf("%d -> ", temp->vertex);

            temp = temp->next;

        }

        printf("\n");

    }

}

// DFS traversal

void DFS(struct Graph* graph, int startVertex) {

    struct Node* adjList = graph->adjLists[startVertex];

    struct Node* temp = adjList;

```

```

graph->visited[startVertex] = 1;

printf("Visited %d \n", startVertex);

while (temp != NULL) {

    int connectedVertex = temp->vertex;

    if (graph->visited[connectedVertex] == 0) {

        DFS(graph, connectedVertex);

    }

    temp = temp->next;

}

}

int main() {

    struct Graph* graph = createGraph();

    addEdge(graph, 0, 1);

    addEdge(graph, 0, 2);

    addEdge(graph, 1, 2);

    addEdge(graph, 1, 3);

    addEdge(graph, 2, 3);

    printf("Graph:\n");

    printGraph(graph);

    printf("\nDFS Traversal starting from vertex 0:\n");

    DFS(graph, 0);

    return 0;

}

```

OUTPUT:

Graph:

2 -> 1 ->

3 -> 2 -> 0 ->

3 -> 1 -> 0 ->

2 -> 1 ->

DFS Traversal starting from vertex 0:

Visited 0

Visited 2

Visited 3

Visited 1